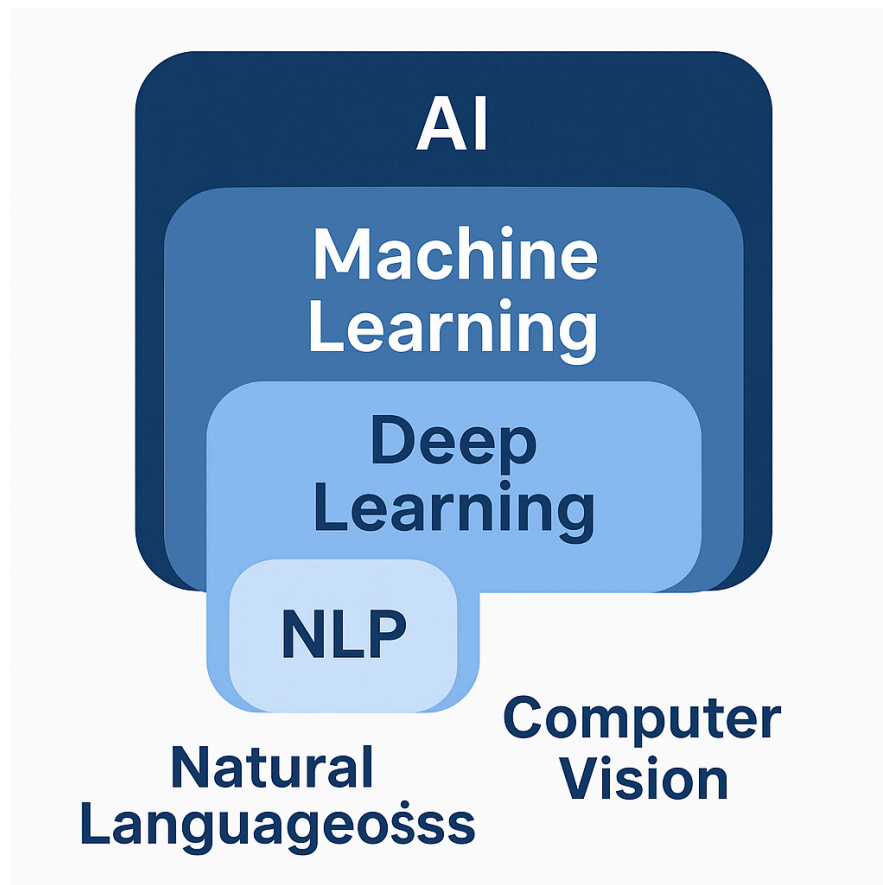




NLP

Real-World Applications of NLP

Application Area	Description / Use	How It Works (NLP Concept Used)
1. Customer Service (Chatbots)	Chatbots handle ~70% of customer queries, improving efficiency and response time.	Uses Intent Recognition and Named Entity Recognition (NER) to understand user input and generate meaningful responses using dialogue management and language generation .

2. Content Moderation	NLP models filter harmful or toxic content with ~98% accuracy to ensure online safety.	Uses Text Classification and Sentiment Analysis models trained on large datasets to detect hate speech, abuse, or spam.
3. Medical Records Analysis	Automates the analysis of medical data, saving ~50% of clinicians' time.	Uses Information Extraction and Entity Linking to identify patient names, diseases, medicines, and symptoms from unstructured text.
4. Legal Document Review	Speeds up legal document review by ~80%, improving accuracy and efficiency.	Uses Text Summarization and Keyword Extraction to identify clauses, entities, and important sections within lengthy legal documents.
5. Market Sentiment Analysis	Helps investors understand trends by analyzing social media and market data.	Uses Sentiment Analysis and Opinion Mining to classify market mood (positive, negative, neutral) from financial news and social media posts.

1 Lexical Preprocessing (Word Level)

Definition:

Lexical preprocessing involves breaking text into words (tokens) and cleaning them for further analysis.

Main Goal:

To normalize text and reduce noise at the **word level**.

Common Techniques:

- **Tokenization:** Breaking text into individual words or sentences.
Example: "I love NLP" → ["I", "love", "NLP"]
- **Stop-word Removal:** Removing common words like "is", "a", "the", etc.
- **Stemming:** Cutting words to their base form.
Example: "Playing", "Played" → "Play"
- **Lemmatization:** Converting words to their dictionary form using grammar rules.
Example: "Better" → "Good"
- **Lowercasing, Removing Punctuation, Removing Numbers, etc.**

Example Sentence:

"Cat is sitting on a mat."

→ After lexical preprocessing → ["cat", "sitting", "mat"]

2 Syntactical Preprocessing (Sentence Level)

Definition:

Syntactical preprocessing ensures that sentences follow **proper grammatical structure**.

Main Goal:

To understand how words are arranged to form meaningful sentences.

Techniques Used:

- **POS Tagging (Part of Speech Tagging):** Identifies nouns, verbs, adjectives, etc.
Example: "She plays football." → [She (PRON), plays (VERB), football

(NOUN)]

- **Parsing:** Analyzes sentence structure (subject–verb–object relationships).
- **Dependency Parsing:** Finds how words depend on each other in a sentence.

Example:

Correct: “He runs fast.”

Incorrect: “He run fast.”

3 Semantic Preprocessing (Meaning Level)

Definition:

Semantic preprocessing focuses on **understanding the meaning** and **context** of words or sentences.

Main Goal:

To extract actual meaning rather than just word structure.

Techniques Used:

- **Named Entity Recognition (NER):** Identifies entities like names, places, or organizations.
Example: “Apple launched a new iPhone in California.” → [Apple = Organization, California = Location]
- **Word Sense Disambiguation:** Determines which meaning of a word is used in context.
Example: “Bank” → (River bank or financial bank?)
- **Coreference Resolution:** Finds what pronouns refer to.
Example: “Raj loves his car. He washes it every day.” → “He” → “Raj”, “it” → “car”.

4 Pragmatic / Discourse Level Preprocessing (Context Level)

Definition:

Pragmatic or discourse preprocessing deals with the **context, intent, and tone** of text across sentences or conversation.

Main Goal:

To interpret **how meaning changes depending on context or user intent**.

Techniques Used:

- **Dialogue Act Recognition:** Identifies if a statement is a question, command, or request.
- **Sarcasm Detection / Sentiment Context:** Understands tone and implied meaning.
Example: “Oh great, another meeting!” → negative sentiment, even though “great” is a positive word.
- **Context Tracking:** Maintains meaning across multiple sentences.

Summary Table

Type of Preprocessing	Level	Focus / Goal	Common Techniques
Lexical	Word level	Cleaning and normalization	Tokenization, Stop-word removal, Stemming, Lemmatization
Syntactical	Sentence level	Grammar and structure	POS tagging, Dependency parsing
Semantic	Meaning level	Understanding context and entities	NER, Word sense disambiguation, Coreference resolution
Pragmatic / Discourse	Context level	Intent, tone, and conversation flow	Dialogue act recognition, Sarcasm detection, Context tracking

Key Takeaway

In theory, NLP preprocessing is often divided into **Lexical** and **Syntactical** types.

But in real-world NLP, we also include **Semantic** and **Pragmatic** preprocessing for deeper language understanding.

■ Lexical Preprocessing Steps (Cleaning and normalization)

Lexical preprocessing deals with **cleaning and preparing text at the word level** before analysis.

It helps remove noise and make the text machine-readable.

Step	Task	Why It Comes Here	Example
1 Tokenization	Split text into words/tokens	We need tokens before we can modify them.	"I love NLP!" → ["I", "love", "NLP", "!"]
2 Lowercasing	Convert all words to lowercase	So that "The" and "the" are treated the same.	["i", "love", "nlp", "!"]
3 Removing Punctuation	Remove symbols like . , ! ?	These don't affect meaning (usually).	["i", "love", "nlp"]
4 Removing Numbers	Remove digits like 2025, 100	Numbers often don't add meaning (task dependent).	"I have 2 cats" → "I have cats"
5 Stop-word Removal	Remove common function words (<i>is, the, and, a, to, of, in</i>)	Comes after punctuation and number removal because stop words are actual words — not symbols or digits.	["i", "love", "the", "nlp"] → ["love", "nlp"]
6 Lemmatization / Stemming	Reduce words to their base form	Final cleanup step to unify similar word forms.	["playing", "played"] → ["play"]

Aspect	Stemming	Lemmatization
Definition	Reduces a word to its root form by chopping off suffixes or prefixes (without understanding grammar).	Converts a word to its base or dictionary form (lemma) by analyzing grammar and meaning .
Method Used	Uses rule-based cutting of word endings.	Uses vocabulary + morphological analysis (WordNet or SpaCy) .
Output Type	May produce non-meaningful or truncated words .	Always produces real, meaningful words .
Example	"Playing", "Played", "Plays" → "Play" ✓ "Studies" → "Studi" ✗	"Playing", "Played", "Plays" → "Play" ✓ "Studies" → "Study" ✓
Speed	Very fast (simple rules).	Slower (does grammatical lookup).
Accuracy	Less accurate (no context awareness).	More accurate , context-aware.
Tool Example	PorterStemmer , SnowballStemmer (in NLTK).	WordNetLemmatizer (NLTK), spaCy lemmatizer.
When to Use	When you need speed over accuracy (e.g., search engines).	When you need clean, meaningful text (e.g., deep learning NLP tasks).

💬 In Simple Words:

- 🪓 **Stemming = Cutting** (rough and fast)
- 🔍 **Lemmatization = Understanding** (accurate and meaningful)

■ Syntactical Preprocessing (Grammar and Structure)

Syntactical preprocessing ensures that sentences follow **proper grammatical structure**.

It focuses on the **relationship between words, sentence formation, and subject–verb agreement**.

This helps NLP models understand **how words are arranged** to create meaningful sentences.

Step	Task	Why It Comes Here	Example
■ 1 Part-of-Speech h (POS) Tagging	Identifies the grammatical role of each word (noun, verb, adjective, etc.).	Helps the system understand sentence structure and how words function grammatically.	Sentence: <i>“The cat sat on the mat.”</i> POS Tags → [('The' , 'DET') , ('cat' , 'NOUN') , ('sat' , 'VERB') , ('on' , 'ADP') , ('the' , 'DET') , ('mat' , 'NOUN')]
■ 2 Dependency Parsing	Analyzes how words depend on each other in a sentence.	Builds a tree-like structure showing which words modify or depend on others.	<i>“The cat sat on the mat.”</i> → ‘cat’ is subject of ‘sat’, ‘mat’ is object of ‘on’
■ 3 Constituency Parsing	Breaks a sentence into sub-phrases (noun phrase, verb phrase, etc.).	Helps identify phrase-level grammar and structure for language generation or correction.	<i>“The quick brown fox jumps over the lazy dog.”</i> → NP (The quick brown fox), VP (jumps over the lazy dog)



Subject–Verb Agreement Check

Checks whether the subject and verb agree in number and tense.

Ensures **grammatical correctness**, important for grammar correction or text validation systems.



“*She runs fast.*”



“*She run fast.*”

✓ Key Points

- Focuses on **sentence grammar and syntactic structure**, not on meaning.
- Provides the foundation for **semantic analysis**, translation, and chatbot grammar.
- Typically done using libraries like **spaCy**, **NLTK**, or **Stanza**.

■ TF-IDF (Term Frequency – Inverse Document Frequency)

Definition:

TF-IDF is a technique to find **important words** in a document by giving more weight to terms that appear frequently in a specific document but more rarely across the entire corpus. [GeeksforGeeks+2Amir Masoud Sefidian - Sefidian Academy+2](#)

When It Comes / Where It Fits:

- It's used **after preprocessing** (tokenization, cleaning, stop-word removal, etc.).
- It is part of **Text Representation / Feature Extraction**, converting cleaned text into numeric features.

- Suitable when you need a representation that emphasises words specific to each document while reducing the influence of common words.
-

When to Use:

- When you need a vector representation of text for models (e.g., classification, clustering).
 - When simple frequency (like Bag of Words) isn't enough and you want to down-weight very common terms and highlight more meaningful ones.
 - Great for tasks like document classification, keyword extraction, search & retrieval. [Built In+1](#)
-

How It Works (in Simple Terms):

- **Term Frequency (TF):** How often a word appears in a document.
 - **Inverse Document Frequency (IDF):** How common or rare a word is across all documents (rare words get higher weight). codingthepast.com
 - **TF-IDF score = $TF \times IDF$** — so a high score means a word appears often in a document but not in many documents overall. [Wikipedia+1](#)
-

Example:

Sentences (documents):

Doc1: "Data science is fun and data science is exciting."

Doc2: "Python makes data science easy and fun."

Vocabulary might be: ["data", "science", "is", "fun", "exciting", "python", "makes", "easy", "and"]

- The word “**data**” appears frequently in both docs → TF high but IDF lower (because appears in many docs) → moderate TF-IDF.
- The word “**exciting**” appears frequently in Doc1 but not in Doc2 → TF high, IDF high → higher TF-IDF weight for Doc1.

Python Library / Function:

- In Python, you can use `sklearn.feature_extraction.text.TfidfVectorizer` to apply TF-IDF easily. [Scikit-learn+1](#)

Example:

```
from sklearn.feature_extraction.text import TfidfVectorizer

sentences = ["Data science is fun", "Python makes data science easy"]

vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(sentences)

print(vectorizer.get_feature_names_out())

print(X.toarray())
```

-

Note (Important):

TF-IDF is **better than plain Bag of Words** for capturing word importance, but it still does **not capture context or word order** — techniques like Word2Vec or BERT are needed for that.

✓ TF-IDF Formula (in words):

$$TF\text{-}IDF = TF \times IDF$$

- High TF → word is frequent in this document
 - High IDF → word is rare across all documents
 - Both high → **important word**
- ♦ High TF + High IDF = Important word (High TF-IDF)
 - ♦ High TF + Low IDF = Common word (Low TF-IDF)

Term Frequency (TF) – Measures how often a word appears in a document.

$$TF = \frac{\text{Number of times the word appears in the document}}{\text{Total words in the document}}$$

Inverse Document Frequency (IDF) – Reduces the weight of common words across documents.

$$IDF = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents containing the word}} \right)$$

TF-IDF Score – Computed by multiplying TF and IDF:

$$TF\text{-}IDF = TF \times IDF$$

$$IDF = \log(\text{Total documents} / \text{Number of documents containing the word})$$

Word	Appears In (Docs)	IDF Value	Importance
“the”	in 100 out of 100 docs	$\log(100/100) = \log(1) = 0$	✗ Very low
“dog”	in 10 out of 100 docs	$\log(100/10) = \log(10) = 1$	● Medium
“daughter”	in 1 out of 100 docs	$\log(100/1) = \log(100) = 2$	● Very high

✓ So the **fewer documents** a word appears in,
the **higher its IDF** → meaning it's more **unique and important**.

Library	Import Example	Purpose / Use in NLP
nltk	<pre>import nltk</pre>	Natural Language Toolkit — used for text preprocessing like tokenization, stopword removal, stemming, and lemmatization.
nltk.tokenize	<pre>from nltk.tokenize import word_tokenize, PunktSentenceTokenizer, sent_tokenize</pre>	Breaks text into words or sentences (tokenization).
nltk.corpus.stopwords	<pre>from nltk.corpus import stopwords</pre>	Provides a list of common English stopwords to remove from text (like <i>the</i> , <i>is</i> , <i>and</i>).

nltk.stem	<code>from nltk.stem import PorterStemmer, WordNetLemmatizer</code>	Used for reducing words to their root or base form (stemming & lemmatization).
sklearn.feature_extraction.text	<code>from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer</code>	Converts text into numerical form using Bag of Words or TF-IDF.
tensorflow.keras.preprocessing.text	<code>from tensorflow.keras.preprocessing.text import Tokenizer</code>	Converts text into integer sequences for deep learning models.
tensorflow.keras.preprocessing.sequence	<code>from tensorflow.keras.preprocessing.sequence import pad_sequences</code>	Pads sequences to make all inputs the same length for RNN/LSTM models.
tensorflow.keras.models	<code>from tensorflow.keras.models import Sequential</code>	Builds deep learning models layer by layer.

tensorflow.keras.layers	<pre>from tensorflow.keras.layers import Embedding, SimpleRNN, Dense</pre>	Adds neural network layers like embedding, RNN, and dense layers.
tensorflow.keras.optimizers	<pre>from tensorflow.keras.optimiz ers import Adam</pre>	Optimizer used for efficient model training.

In Simple Words

 **NLTK & sklearn** → For **text preprocessing and feature extraction**
 **TensorFlow / Keras** → For **building and training deep learning models** in NLP

Resource	Used For	Needed When
'punkt'	Tokenization	Always
'stopwords'	Stopword removal	Always
'wordnet'	Lemmatization	Always
'averaged_perceptron_tagger'	POS tagging	Only if doing syntactical analysis

Feature	<code>sent_tokenize()</code>	<code>PunktSentenceTokenizer()</code>
Definition	A <i>pre-trained</i> sentence tokenizer that comes ready to use with NLTK.	A <i>trainable</i> sentence tokenizer that you can train on your own data.
Training	Already trained on large English text corpus.	Can be trained on new text using <code>.train()</code> to adapt to your writing style or domain.
Accuracy	Generally more accurate for most text since it uses a pre-trained model.	Might break sentences incorrectly if trained on a very short or domain-specific text.
Example Behavior	Correctly treats “Dr.” and “U.S.” as abbreviations (doesn’t split sentences incorrectly).	Because you trained it on the <i>same small text</i> , it learns that “Dr.” ends a sentence → splits incorrectly.
Use Case	Use when working with normal English sentences.	Use when you want to train a tokenizer for a specific writing style (like medical reports, legal text, etc.).

Aspect	Bag of Words (BoW)	TF-IDF	Word Embeddings
Representation	Words as individual counts	Word frequency adjusted by rarity	Dense vector representation
Context Understanding	No understanding of context	No understanding of context	Captures context and relationships between words
Vector Size	High-dimensional, sparse	High-dimensional, sparse	Low-dimensional, dense
Word Similarity	Doesn’t capture word similarity	Doesn’t capture word similarity	Words with similar meanings are closer in vector space

Computation	Simple and fast	Requires computation of IDF	Requires training on large datasets
--------------------	-----------------	-----------------------------	-------------------------------------



Conclusion

- **Bag of Words (BoW)** and **TF-IDF** are simple yet limited methods for representing text.
They rely on word counts or frequencies and **do not capture the meaning or relationships** between words.
While effective for basic NLP tasks, they fail to understand context or semantics.
 - **Word Embeddings**, on the other hand, offer a **more advanced and meaningful representation** of text.
They encode both the **semantic meaning** and **contextual relationships** between words,
making them far more powerful for modern NLP tasks such as **sentiment analysis, translation, chatbot systems, and recommendation engines**.
-



Steps for Text Classification (NLP Pipeline)

1. Collecting Data

- Gather text data from sources like:
 - Social media posts, emails, reviews, articles, etc.
 - Example: Tweets for sentiment analysis (positive/negative/neutral).
-

2. Labeling Data

- Each text must have a **label (category)** in supervised learning.
- Example:

- “I love this product” → Positive
 - “This is terrible” → Negative
 - Labeling can be **manual** or **automated** (if dataset already labeled).
-

3. Preprocessing the Text

Clean and prepare the text for modeling.

Steps include:

Tokenization: Split text into words/sentences.

```
from nltk.tokenize import word_tokenize  
tokens = word_tokenize(text)
```

- - **Lowercasing:** Convert text to lowercase.
 - **Removing Stopwords:** Remove common words like *the, is, in*.
 - **Lemmatization/Stemming:** Convert words to root form (e.g., “running” → “run”).
 - **Removing Special Characters/Numbers:** Remove unwanted symbols and digits.
-

4. Vectorization (Convert Text to Numbers)

Machine learning models only understand numbers.

You can use one of the following techniques:

Technique	Description
Bag of Words (BoW)	Counts word occurrences. Simple but ignores meaning.
TF-IDF	Weighs words by importance across documents.

Word Embeddings (Word2Vec, GloVe, SentenceTransformer)

Captures **semantic meaning** and **context** between words/ Word2Vec captures relationships between words.

(SentenceTransformer or Word2Vec are best for deep learning and modern NLP.)

Examples

1) Bag of Words (BoW)

Example

Document: "I love dogs"

Vocabulary = [I, love, dogs]

BoW Vector = [1, 1, 1]

(Each number = count of that word)

When to use

- Simple NLP tasks
- No need for meaning or context
- Small datasets
- Basic Logistic Regression / Naive Bayes models

2) TF-IDF (Term Frequency – Inverse Document Frequency)

Here is the *most crystal-clear example with two documents* 📌

Two Documents

Doc 1: "I love machine learning"

Doc 2: "Machine learning is powerful"

Let's calculate TF-IDF for "powerful" and "machine".

◆ Example 1: Word "powerful" → HIGH TF-IDF

TF (Term Frequency):

- In Doc 1 → 0
- In Doc 2 → 1

IDF (Inverse Doc Frequency):

Total docs = 2

Word appears in 1 document.

$$IDF = \log\left(\frac{2}{1}\right) = \log(2)$$

➡ **IDF is high** (word is rare)

TF-IDF = TF × IDF

- Doc 1: $0 \times \log(2) = 0$
- Doc 2: $1 \times \log(2) = \text{high score}$

✓ **Interpretation**

➡ "powerful" is **important** and **unique** in Document 2.

◆ Example 2: Word "machine" → LOW TF-IDF

TF

- $Doc\ 1 \rightarrow 1$
- $Doc\ 2 \rightarrow 1$

IDF

Word appears in **both documents**:

$$IDF = \log\left(\frac{2}{2}\right) = \log(1) = 0$$

→ IDF is 0 (word appears everywhere → NOT important)

TF-IDF

- $1 \times 0 = 0$

✓ Interpretation

→ “machine” is **not unique**, appears everywhere → low importance.

When to use TF-IDF

- Document classification
 - Search engines
 - Keyword extraction
 - When you want **important words**, not common words
-

✓ 3) Word2Vec (Embeddings)

Example

Word2Vec produces vectors like:

- “king” → [0.52, -0.18, 0.44, ...]
- “queen” → **similar vector**

It captures relationships:

king - man + woman = queen
king - man + woman = queen

When to use

- Chatbots
 - Semantic search
 - Intent detection
 - When **meaning**, **context**, and **similarity** are important
-

Feature	Skip-Gram (sg = 1)	CBOW (sg = 0)
What it predicts?	Predicts context words (nearby words) from a target word	Predicts target word from context words (nearby words)
Training pairs look like	(target → context)	(context → target)
Best for	Small datasets	Large datasets
Rare word performance	✓ Very good	✗ Not very good
Training speed	✗ Slower	✓ Faster

Noise handling	✗ Less smooth	✓ Smooths noise
Vector quality	✓ Higher-quality semantic embeddings	✓ Good embeddings but less detailed
Focus	Learns detailed relationships	Learns general meaning
Real-world analogy	"Given one word, guess the words around it."	"Given surrounding words, guess the missing middle word."

5. Model Selection and Building

Choose a model for classification.

Common algorithms:

- **Naive Bayes** → Best for small datasets, spam detection.
 - **Support Vector Machine (SVM)** → Works well for text classification.
 - **Deep Learning Models** → CNN, RNN, LSTM for complex NLP tasks.
-

6. Training the Model

Split data into **Training** and **Test** sets.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2)
```

- Train model on labeled training data.
 - Tune hyperparameters for better accuracy.
-

7. Evaluating the Model

After training, check how well your model performs.

Common evaluation metrics:

- **Accuracy:** (Correct predictions / Total predictions)
 - **Precision:** Correctly predicted positives / All predicted positives
 - **Recall:** Correctly predicted positives / All actual positives
 - **F1-Score:** Balance between Precision and Recall
-

✅ Summary: Full Text Classification Workflow

1. **Collect Data**
2. **Label Data**
3. **Preprocess Text** (cleaning, tokenization, stopwords, lemmatization)
4. **Convert Text → Numbers** (TF-IDF / Word Embeddings)
5. **Choose & Train Model** (Naive Bayes / SVM / Deep Learning)
6. **Evaluate Performance** (Accuracy, Precision, Recall, F1-Score)

✅ Standard NLP Preprocessing Order

- 1 **Lowercasing**
- 2 **Remove punctuation / special characters**
- 3 **Tokenization**

4 Remove stopwords

5 Lemmatization (or stemming)